

Bayesian Optimisation (General)

Bayesian Optimisation is not a specific algorithm, but a **strategy** for global optimisation. It relies on two coupled mathematical components working in a loop:

1. A **Probabilistic Model** (The Surrogate): Describes what the function *might* look like.
2. An **Acquisition Function** (The Decision Rule): Calculates the "utility" of sampling any given point based on that description.

The process is governed by **Bayes' Theorem**. A "Prior" belief (the initial guess of the function's shape) is updated with "Evidence" (new simulation data) to produce a "Posterior" belief (the updated model).

1. The Probabilistic Model (The Map)

Instead of a single line of best fit, a probability distribution is defined over the function. For every point x in the design space, the model provides two values:

- $\mu(x)$: The **Mean** (The expected value of the objective, e.g., TBR).
- $\sigma(x)$: The **Variance** (The uncertainty or "error bar" at that point).

Key Mechanism: Near observed data points, $\sigma(x)$ collapses to zero (we know the value). Far from data points, $\sigma(x)$ increases (we are uncertain).

2. The Acquisition Function (The Compass)

This is the mathematical mechanism that drives the search. It takes the Mean (μ) and Variance (σ) from the model and combines them into a single **Utility Score** for every possible design point.

This can then tell us which point to explore next!

A common example is **Upper Confidence Bound (UCB)**. The utility $\alpha(x)$ is calculated as:

$$\alpha(x) = \mu(x) + \kappa \cdot \sigma(x)$$

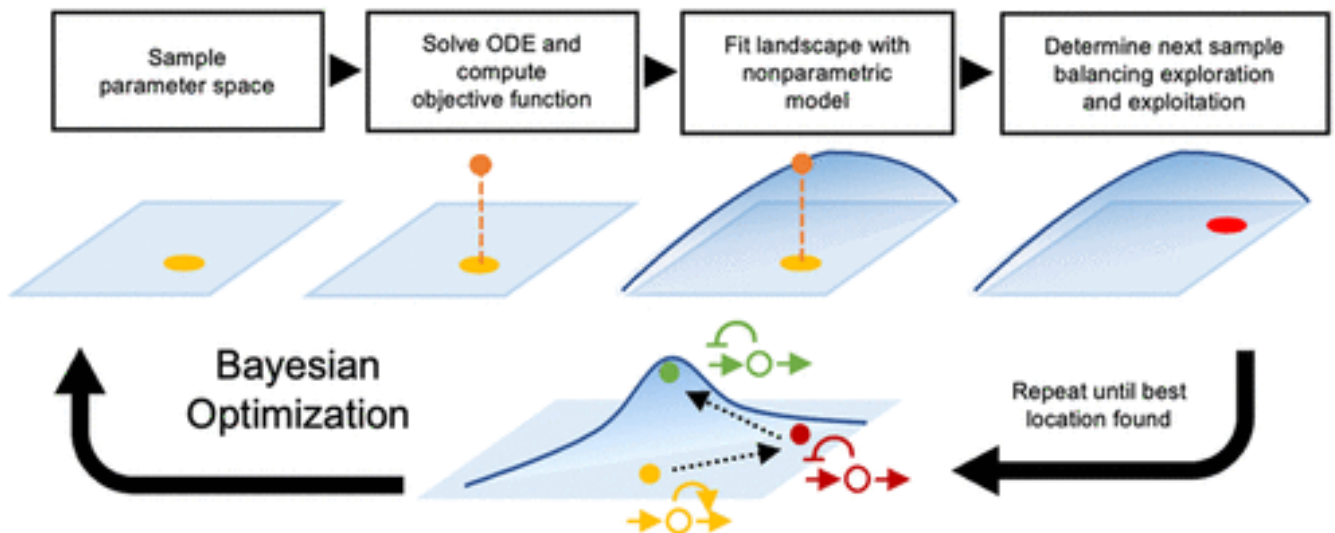
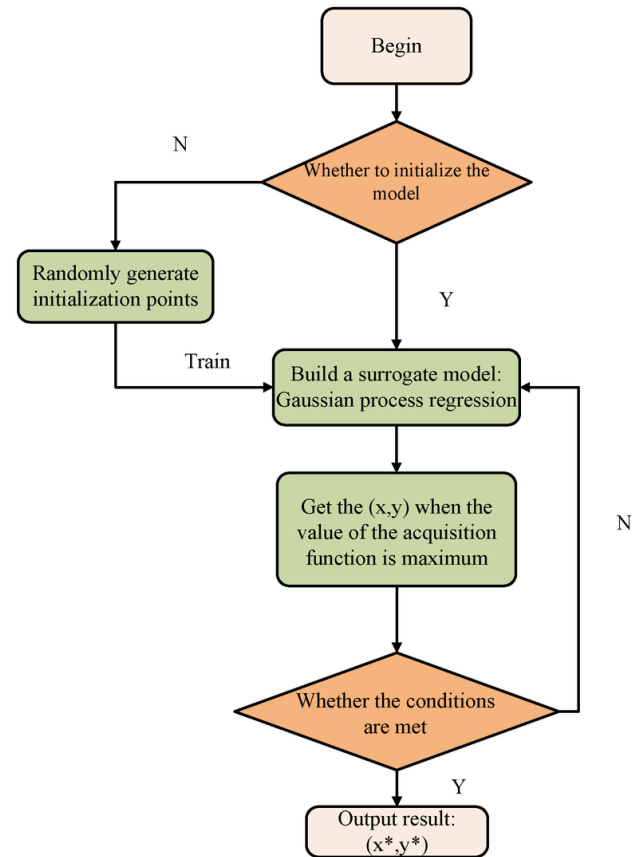
- $\mu(x)$ encourages **Exploitation** (visiting high peaks).
- $\sigma(x)$ encourages **Exploration** (visiting high uncertainty).
- κ is a tunable parameter deciding how much to prioritise exploration.

The Step-by-Step Workflow

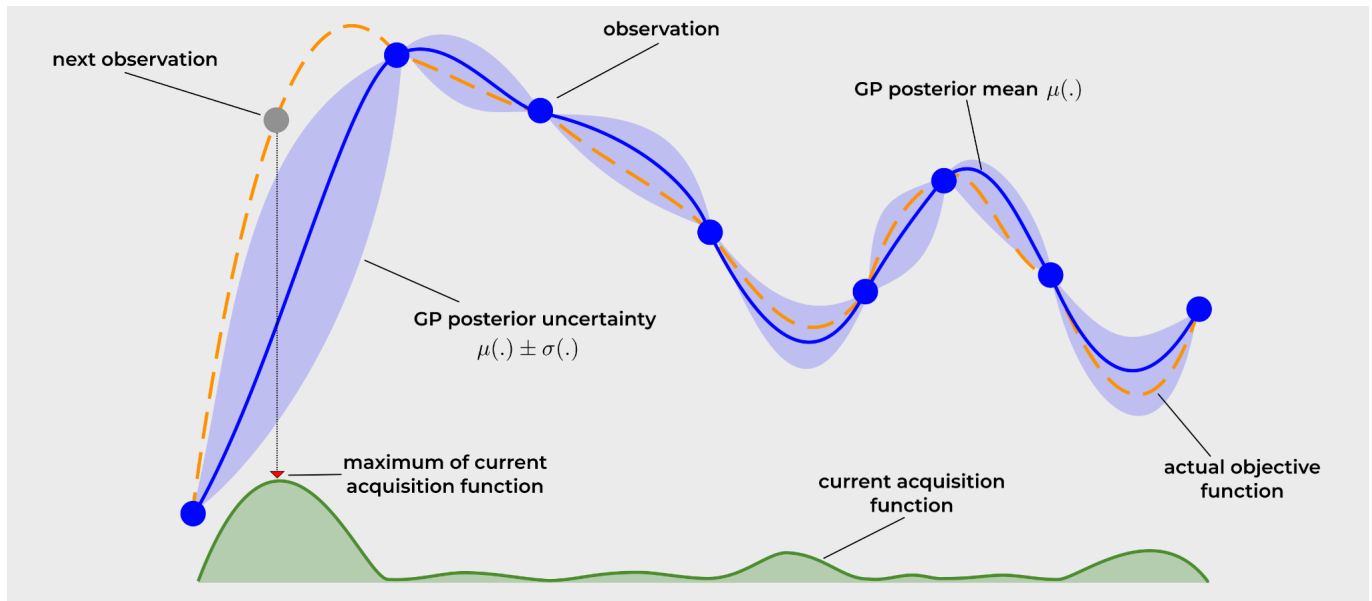
1. **Initial Sampling:** A small set of random design points is evaluated using the expensive objective function (OpenMC).
2. **Model Fitting:** A surrogate model (e.g., Gaussian Process) is trained on this data. A mean and variance are calculated for the entire un-sampled design space.
3. **Acquisition Optimisation:** The Acquisition Function (e.g., UCB) is calculated across the space. The peak of *this* function is found using a standard cheap optimiser. This peak

represents the point with the highest potential value (either because it is likely good, or because the uncertainty is so high it is worth checking).

4. **Evaluation:** The expensive simulation is run at the coordinates identified in Step 3.
5. **Posterior Update:** The new data point is appended to the dataset. The model is mathematically updated:
 - The mean shifts to include the new point.
 - The uncertainty at that point drops to zero.
 - The uncertainty in the surrounding region is reduced.
6. **Iteration:** Steps 3-5 are repeated until convergence (no further improvement is found).



Visualising the "Low Level" Update



- *Top Graph:* The dashed orange line is the real function (unknown). The solid blue line is the model's guess (μ). The shaded blue area is the uncertainty (σ).
- *Bottom Graph:* The Acquisition Function. Note how it peaks where there is a combination of high predicted value AND high uncertainty. The next sample is taken at the peak of this green curve.

Surrogate Modelling: Gaussian Process Regression

1. Surrogate Modelling: The Concept

A Surrogate model is essentially a replacement function for a complex task (like a simulation) that can quickly get the desired output from the same set of inputs.

A surrogate model (also known as a metamodel or emulator) functions as a computationally efficient mathematical approximation of a complex, high-fidelity simulation.

- **The Problem:** The primary physics solver, OpenMC, requires significant computational resources ($10^5 - 10^7$ particles) to converge a single design point. Performing a high-resolution grid search or optimisation on this "expensive" function is intractable, particularly as the move to heterogeneous geometry increases simulation costs.
- **The Solution:** The surrogate model replaces the expensive "black box" (OpenMC) within the optimisation loop. It provides near-instantaneous predictions of the objective values (e.g., TBR, Heating) for any given set of input parameters.
- **The Trade-off:** Accuracy is sacrificed for speed. The surrogate is trained on a limited set of high-fidelity data points and interpolates between them.

2. Gaussian Process Regression (GPR)

Gaussian Process Regression is the specific class of surrogate model selected for this project. Unlike standard regression (e.g., polynomial curve fitting) which assumes a specific function shape (like $y = mx + c$), GPR is a non-parametric Bayesian approach. It does not fit a single line; instead, it defines a probability distribution over *all possible functions* that could fit the observed data.

The Mechanism

A Gaussian Process is defined by two key mathematical components:

- **The Mean Function $\mu(x)$:** The "best guess" prediction for the objective value at any point x .
- **The Covariance Function (Kernel) $k(x, x')$:** This is the core "low-level" mechanism of the GP. It defines the similarity between two points in the design space. The assumption is that points which are close in input space (e.g., two reactor designs with very similar blanket thicknesses) will have correlated outputs (similar TBR).

The Workflow: From Prior to Posterior

1. **The Prior:** Before any simulations are run, a "Prior" distribution is defined. This represents general assumptions about the physics (e.g., that the TBR response is smooth and continuous).
2. **The Kernel (Hyperparameters):** The specific behaviour of the function is controlled by the Kernel's hyperparameters:
 - *Length Scale:* Determines how "wiggly" the function is. A short length scale means TBR changes rapidly with small geometric tweaks; a long length scale implies a smooth, slow change.
 - *Signal Variance:* The vertical amplitude of the function.
 - *Noise Variance:* Accounts for the statistical error inherent in the Monte Carlo method.
3. **The Posterior Update:** As OpenMC data points are observed, the GP is "conditioned" on this data.
 - At the observed points, the probability distribution "squeezes" tight (uncertainty $\sigma \rightarrow 0$ or the noise floor).
 - Between points, the distribution flares out, representing high uncertainty.
 - This updated distribution is the **Posterior**.

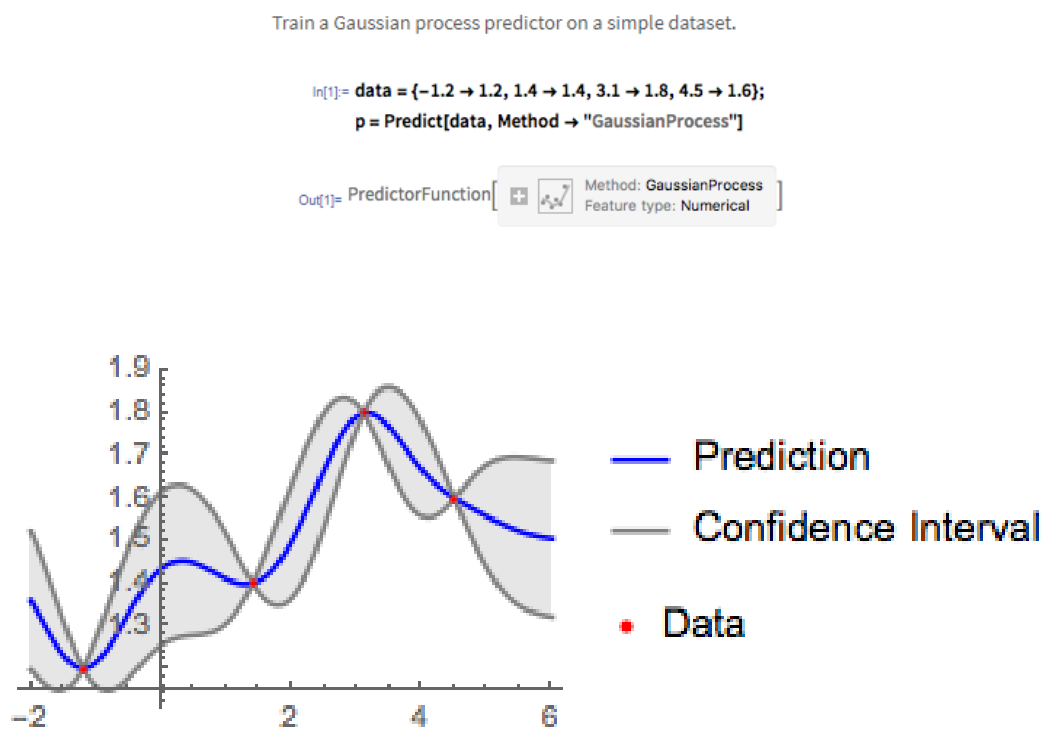
3. Why GPR for Fusion Neutronics?

GPR is uniquely suited for this optimization task compared to other surrogates (like Neural Networks) for three reasons:

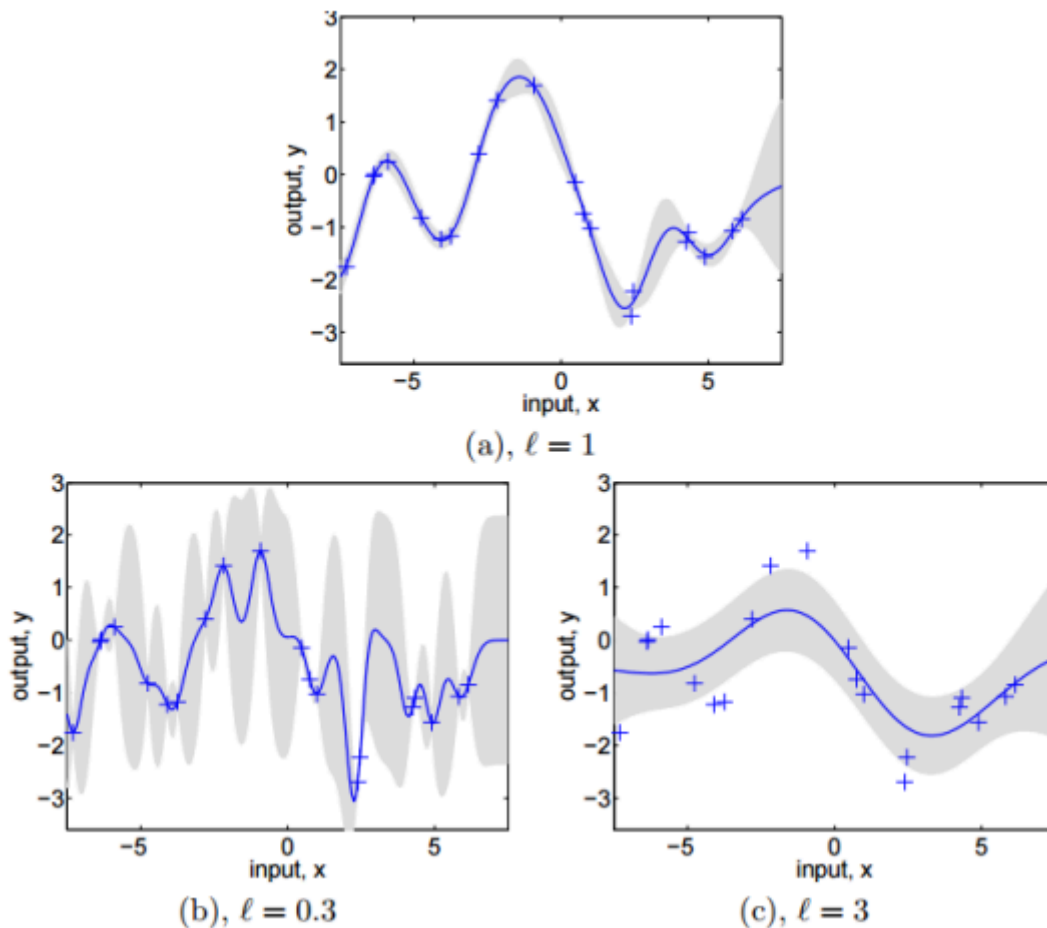
- **Small Data Regime:** Neural networks require thousands of data points to train. GPR is effective with very few samples (e.g., <50 simulations), which matches the high cost of the OpenMC solver.

- **Uncertainty Quantification (UQ):** GPR inherently outputs a variance (σ) alongside the prediction. This variance is the direct input for the Acquisition Function (discussed in previous notes), enabling the "Exploration" half of the strategy.
- **Noise Handling:** The stochastic nature of neutron transport means two identical simulations will produce slightly different results due to statistical error. GPR can explicitly model this "noise term" in the kernel, preventing the optimiser from overfitting to statistical ghosts.

4. Visualising the Kernel's Role & Hyperparameter effects



(Fig 1: general kernel function)



(Fig 2: diagram showing varying the length scale)

- *Interpretation:* The Kernel dictates how information "bleeds" from one data point to its neighbours. If the length scale is set correctly, a single simulation at a blanket thickness of 50cm provides information about the probable TBR at 45cm and 55cm, reducing the number of simulations required.

Acquisition Functions

1. The Concept: Transforming Probability into Utility

The Acquisition Function is the decision-making mechanism within the Bayesian Optimisation loop. While the Surrogate Model (Gaussian Process) provides a probabilistic map of the design space (giving a mean $\mu(x)$ and variance $\sigma(x)$ at every point), it does not essentially dictate *where* to sample next. The Acquisition Function transforms this probabilistic information into a single scalar value called "Utility" or "Acquisition Value."

The point in the design space that maximises this utility function is selected as the next candidate for the expensive OpenMC simulation.

2. The Exploration-Exploitation Trade-off

The core function of the acquisition step is to balance two conflicting objectives:

- **Exploitation:** Focusing on regions where the Surrogate Model predicts a high objective value (high mean μ). This refines the solution around known maxima (e.g., refining the $Li_{17}Pb_{83}$ thickness around the current best estimate).
- **Exploration:** Focusing on regions where the model uncertainty is high (high variance σ). This prevents the optimiser from getting stuck in local optima by investigating unsampled regions of the parameter space.

3. Common Acquisition Functions

Several mathematical formulations exist to quantify this utility:

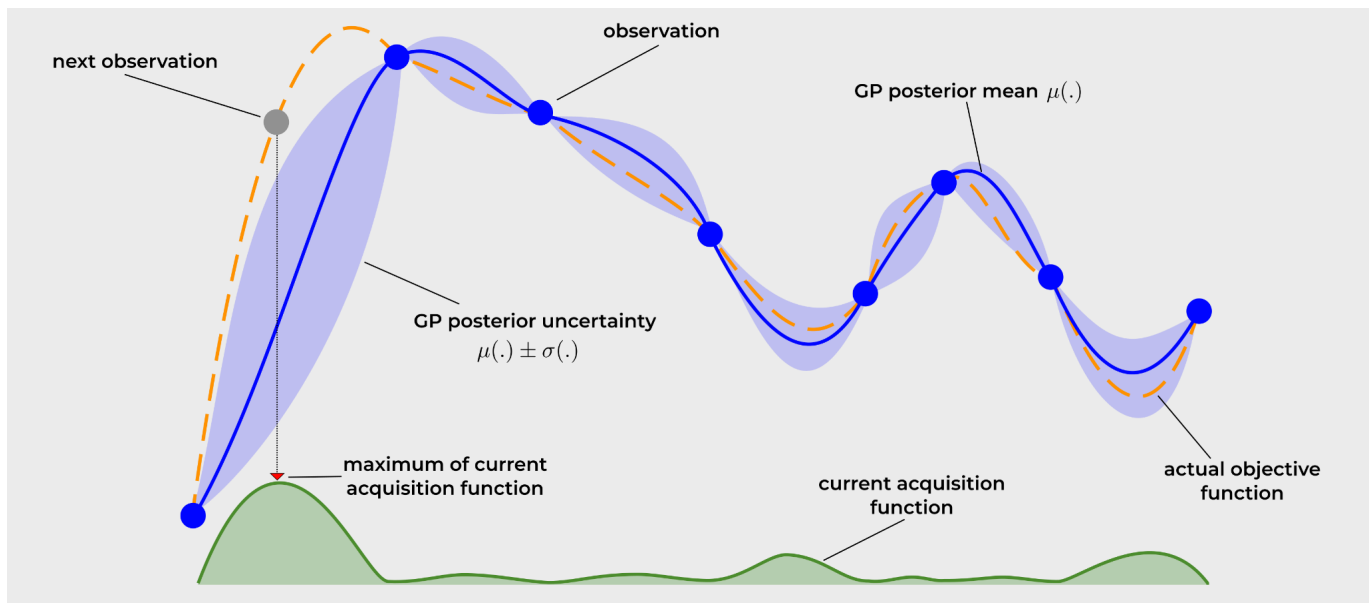
- **Expected Improvement (EI):** The most standard approach. It calculates the expectation of the degree to which a new sample will exceed the current best observation.
 - *Mechanism:* It accounts for both the probability of improvement and the magnitude of that improvement. If a point has a lower mean but massive uncertainty, it might still have a high "Expected Improvement" because the potential upside is large.
- **Upper Confidence Bound (UCB):** A more direct combination of the mean and variance, typically expressed as:

$$\alpha(x) = \mu(x) + \kappa\sigma(x)$$
 - *Mechanism:* The parameter κ is manually tuned to weigh exploration against exploitation. A higher κ forces the algorithm to be more "curious" about unknown regions.
- **Probability of Improvement (PI):** A simpler metric that calculates only the chance that a new point is better than the current best, ignoring *how much* better. This often leads to overly exploitative behaviour and is less commonly used for complex landscapes.

4. Acquisition Optimisation (The "Inner Loop")

A critical distinction must be made between optimising the *reactor* and optimising the *acquisition function*.

- **The Problem:** To find the next sampling point, the maximum of the Acquisition Function itself must be located.
- **The Solution:** Unlike the expensive OpenMC simulation, the Acquisition Function is analytically defined and cheap to evaluate (derived from the Gaussian Process). Therefore, a standard, inexpensive numerical optimiser (such as L-BFGS-B, Direct Search, or a Genetic Algorithm) is used to thoroughly search the Acquisition Function surface.
- **The Workflow:**
 1. The Gaussian Process is frozen.
 2. A secondary optimiser runs thousands of evaluations on the Acquisition Function surface to find its global maximum.
 3. The coordinates of this maximum are returned as the next design point for OpenMC.



- *Interpretation:* The expensive "Black Box" function (top) is sampled only a few times. The cheap Acquisition Function (bottom) is continuously optimised to guide the search. The red triangle indicates the point of Maximum Acquisition—this coordinate is sent to the solver.

Multi-Objective Optimisation & Pareto Fronts **[did not get to in our project]**

1. The Problem: Conflicting Objectives

In complex engineering systems like a fusion reactor, it is rarely possible to optimise a single metric without degrading others. For this design project, the following conflicts are identified:

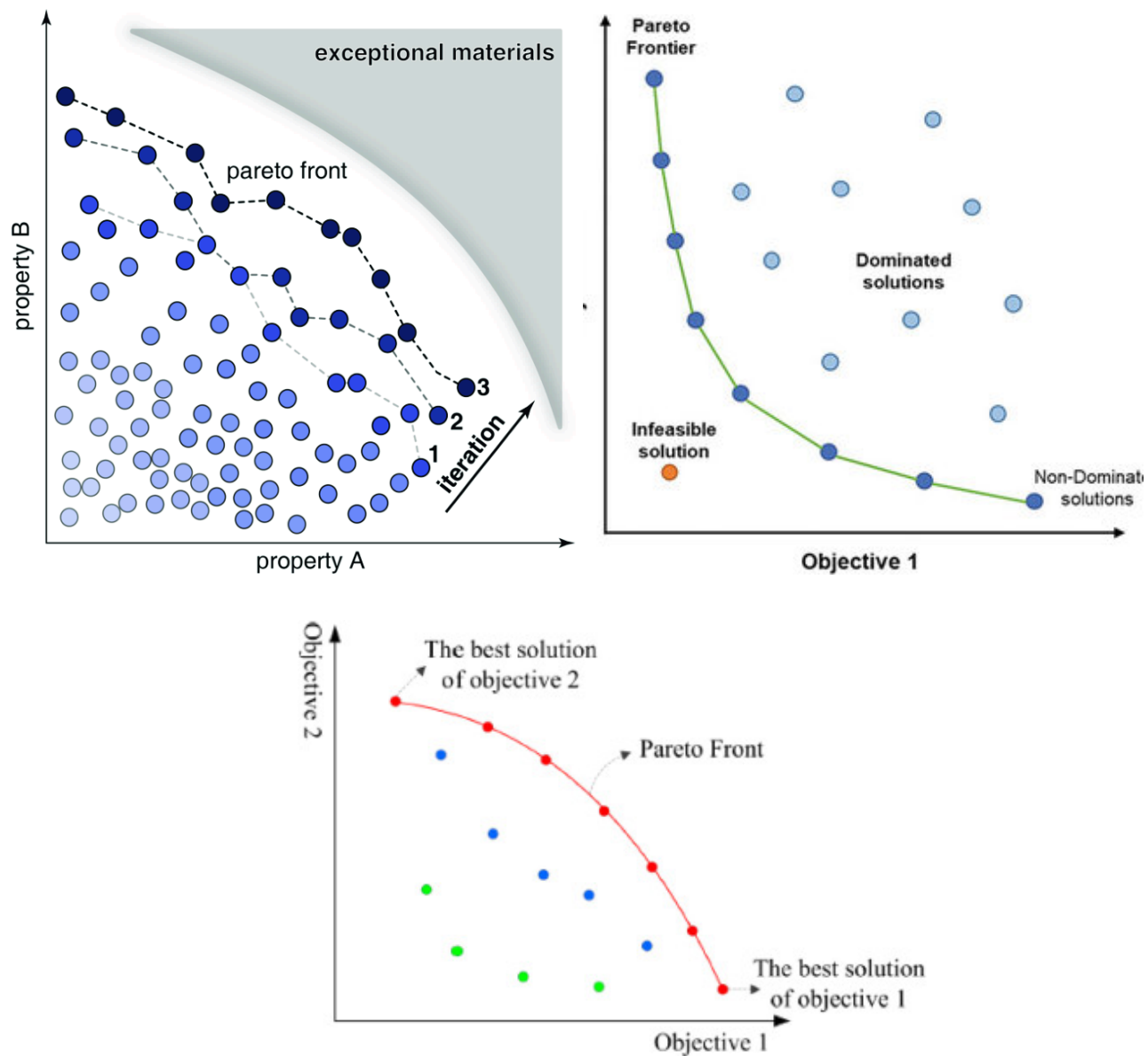
- **TBR vs. Shielding:** Increasing the lead-lithium thickness improves tritium breeding but may reduce the shielding effectiveness for the TF coils or increase the overall reactor size.
- **TBR vs. Safety:** Using water as a coolant/moderator significantly boosts TBR (due to efficient moderation) but introduces a chemical explosion risk (Li-Water reaction).
- **Performance vs. Cost:** High enrichment (90% ^6Li) maximises TBR but drastically increases fuel costs.

2. The Solution: Pareto Optimality

Instead of searching for a single "perfect" design (which does not exist), the goal is to identify a set of "optimal trade-off" designs.

- **Dominance:** A design A is said to "dominate" design B if A is better than B in at least one objective and no worse in any others.
- **Non-Dominated Solutions:** If a design cannot be improved in one objective without making another objective worse, it is "non-dominated."
- **The Pareto Front:** The set of *all* non-dominated solutions forms a curve (or surface in 3D) called the **Pareto Front**.

3. Visualising the Pareto Front



- *Interpretation:* The axes represent two conflicting objectives (e.g., Maximise TBR vs. Minimise Heating).
- **The Curve:** The solid line represents the efficient frontier. Points on this line are "Pareto Optimal."
- **The Feasible Region:** The area below the curve (or above if we want to minimise the objectives) contains all physically possible designs.
- **The Goal:** The optimisation algorithm should push the design points from the grey region (inefficient) up towards the red line (optimal). A design at point *C* is strictly inferior to *A* and *B*. However, between *A* and *B*, there is no mathematical "best"—the choice depends on engineering preference.

4. The Workflow in Bayesian Optimisation

Standard Bayesian Optimisation (which targets a single peak) must be adapted for this multi-objective context.

- **Scalarisation (Weighted Sum):** The simplest approach is to combine objectives into a single score:

$$Score = w_1 \cdot TBR - w_2 \cdot Heating$$

- *Drawback:* This finds only one point on the curve, determined by the arbitrary weights w_1 and w_2 .
- **Pareto-Based Acquisition (EHVI):** A more advanced method uses **Expected Hypervolume Improvement (EHVI)**.
 - *Mechanism:* Instead of seeking the highest peak, the algorithm seeks the point that most expands the "volume" dominated by the current Pareto front. This forces the model to explore the *entire* trade-off curve, giving a range of options rather than a single answer.

5. Application to the Project

For the second semester, this approach allows the presentation of a "family" of reactor designs:

1. **High-Performance Mode:** Max TBR (> 1.20), but higher heating/cost.
2. **Balanced Mode:** Adequate TBR (1.10), safe shielding levels.
3. **Economy/Safety Mode:** Minimized enrichment, minimized hazardous coolants, just meeting TBR targets.

Deep Dive: Acquisition Function

Expected Improvement (EI)

"The Prudent Improver"

Expected Improvement asks a specific probabilistic question: *"Given what I know, how much **better** is this new point likely to be compared to the best result I have already seen?"*

Mathematically, if $f(x)$ is our Gaussian Process prediction (which is a distribution, not a single number) and f^* is our current best observed value, EI calculates:

$$EI(x) = \underbrace{(\mu(x) - f^*)\Phi(Z)}_{\text{Exploitation}} + \underbrace{\sigma(x)\phi(Z)}_{\text{Exploration}}$$

- **How it behaves:** It focuses on **gains**. If a point has high uncertainty (σ) but a very low mean (μ), EI ignores it because the probability of it beating our current best record (f^*) is vanishingly small.
- **The Failure Mode (Rastrigin):**

Once the optimizer found a local peak in the Rastrigin function, that became the new f^* .

 - The "valleys" between peaks have low mean predictions.
 - Even though the uncertainty in those valleys was high, the probability of the value jumping *all the way up* from the valley floor to beat the current peak was tiny.
 - So, EI decided: "It's not worth checking the valleys. I'll just refine the peak I'm already on."

Probabilistic Deep Dive

This is the single most common confusion with Bayesian Optimization. we have understood the "Exploitation" part perfectly, but we missed the **Probabilistic** part.

we are assuming $f(x)$ is a single number (the mean). It is not. In a Gaussian Process, $f(x)$ is a **Probability Distribution** (a Bell Curve).

Here is the visual explanation of how EI explores.

The Scenario

Imagine our current best result is **100** ($f^* = 100$).

The optimizer is looking at a completely unexplored point x_{new} .

- **The Mean (μ):** The model thinks the value at x_{new} is probably around **80**.
- **The Uncertainty (σ):** Because it has never visited this spot, the uncertainty is huge, say ± 30 .

The "Dice Roll"

If we calculate just the mean difference: $80 - 100 = -20$. The optimizer says "This is worse, ignore it." **This is what we were thinking.**

But EI asks: "If I roll the dice on this Bell Curve $N(80, 30)$, is there **any chance** it lands above 100?"

Because the uncertainty is high (30), the Bell Curve is very wide. The "upper tail" of the curve stretches past 100.

- There might be a 15% chance that the actual value is 110.
- There might be a 1% chance the actual value is 140.

EI calculates the area of that upper tail. It sums up all those small probabilities of hitting a jackpot. If the uncertainty (σ) is large enough, that "potential jackpot" value becomes higher than the "sure thing" of staying at a local peak.

The Full Equation (The Proof)

The mathematical formula for EI actually has two distinct terms. This makes it much clearer why it explores.

$$EI(x) = \underbrace{(\mu(x) - f^*)\Phi(Z)}_{\text{Exploitation}} + \underbrace{\sigma(x)\phi(Z)}_{\text{Exploration}}$$

(Where Φ and ϕ are standard normal distribution functions and Z is the normalized score)

- **Term 1 (Exploitation):** This is what we were looking at. It drives the model to points where the Mean $\mu(x)$ is already higher than f^* .

- **Term 2 (Exploration):** Notice this term depends on $\sigma(x)$. If the uncertainty σ is massive, this term becomes huge, even if the mean is low.

So, EI assumes: "I know the average is low, but the error bars are so big that there is a small chance this is the best point in the universe. I'm going to check it."

Why it failed on Rastrigin

-our intuition about it "never exploring" was actually half-right for the Rastrigin function specifically.

Rastrigin has deep valleys.

- Current Best: 10
- Valley Mean: -50
- Uncertainty: ± 5

Even though there is uncertainty, the mean is **so low** that the Bell Curve's tail doesn't even reach the Current Best. The probability of improvement drops to effectively zero.

That is why **UCB (Upper Confidence Bound)** is better for Rastrigin. UCB doesn't care about probability; it just adds the uncertainty to the mean ($\mu + \beta\sigma$) and checks the ceiling.

Upper Confidence Bound (UCB)

"The Optimistic Explorer"

Upper Confidence Bound asks a different question: *"What is the **plausible best-case scenario** for this point?"*

It doesn't care about our current best record (f^*). It simply constructs a ceiling based on the uncertainty:

$$UCB(x) = \mu(x) + \sqrt{\beta} \cdot \sigma(x)$$

- $\mu(x)$: The Mean (Exploitation) — "This looks good."
- $\sigma(x)$: The Uncertainty (Exploration) — "I haven't looked here yet."
- β : The Hyperparameter (The "Optimism" Knob).

Why it works on Rastrigin:

By setting β to a high number (like 6.0), we are telling the model: *"I don't care if the mean prediction is low. If there is uncertainty, assume the value is HUGE and go check it."*

This forces the acquisition function to act like a bulldozer. It sees the deep valleys of the Rastrigin function, sees the high uncertainty in the middle of them, and dives in—not because it expects to win, but because it *might*.

Summary Table

Feature	Expected Improvement (Default)	Upper Confidence Bound (Proposed)
Philosophy	"Beat the current record."	"What is the potential ceiling?"
Risk Profile	Risk Averse (won't explore low-mean areas).	Risk Tolerant (explores anywhere with uncertainty).
Best For	Polishing a solution; Convex problems.	multimodal/chaotic landscapes (like Rastrigin).
our Issue	Got stuck in a local max because it stopped exploring.	Will force exploration of other peaks.

Deep Dive: GPR Covariance

Covariance is one of those mathematical terms that gets thrown around constantly in machine learning, but its physical intuition can sometimes get lost in the code.

At its simplest, **covariance measures how two variables change together**.



Here is a breakdown of what it means in general, why Ax uses it, and how it acts as the secret engine of our fusion reactor surrogate model.

The General Concept

Imagine we are tracking two variables. Covariance tells us about their directional relationship:

- **Positive Covariance:** The variables move together. (e.g., As reactor temperature goes up, plasma pressure tends to go up).
- **Negative Covariance:** The variables move in opposite directions. (e.g., As blanket thickness goes up, available inner vessel volume goes down).

- **Zero Covariance:** The variables have no predictable linear relationship; they move independently.

Why Ax gives us a **covariance** dictionary

Ax is heavily engineered for **multi-objective optimization**. While we are currently only optimizing one metric (**tbr**), Ax is built to handle situations where we might be tracking **tbr**, **heat_flux**, and **tritium_leakage** all at the same time.

If we had multiple metrics, that covariance dictionary would look something like this:

```
{
  'tbr': {'tbr': 0.0025, 'heat_flux': 0.0012},
  'heat_flux': {'tbr': 0.0012, 'heat_flux': 0.0400}
}
```

This matrix tells the optimizer not just how uncertain it is about each individual metric, but **how the uncertainties are linked**. If the model predicts a sudden spike in TBR, the positive covariance (**0.0012**) tells it to expect a spike in heat flux, too.

Covariance vs. Variance

we might be wondering: *"If covariance measures the relationship between TWO variables, why am I looking up the covariance of TBR with itself?"*

In statistics, the covariance of a variable with itself is just its **variance**.

$$Cov(X, X) = Var(X) = \sigma^2$$

By querying `covariance['tbr']['tbr']`, we are asking the model: *"How much does the TBR vary around its own expected mean?"* Taking the square root of that variance gives us the standard deviation (σ), which is the $\pm$ uncertainty bound we are printing in our terminal.

Covariance in Gaussian Processes (our Surrogate Model)

Since we are using BoTorch, there is one more layer to this that is crucial for a master's project.

In a Gaussian Process (GP) surrogate model, covariance is the actual engine that drives the AI's "belief." The GP uses a mathematical function called a **Kernel** (in our code, it looks like it defaults to a Matern kernel). The kernel calculates the covariance between *points in our physical search space*.

If we evaluate a specific breeder thickness, the GP assumes that a thickness just 0.1 cm away will have a very similar TBR. The kernel says those two points have **high covariance**. If we look at a point 10 cm away, the kernel says they have **low covariance** (they are independent).

When our peak-hunt logic looks for "novel" secondary peaks, it is mathematically looking for areas of the search space that have low covariance with the points we have already tested!